LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh: Anggraeni Dwi Zahra
NIM. 2410817220018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : AnggraeniDwi Zahra
NIM : 2410817220018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 19930703 201903 01 011

DAFTAR ISI

DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
B. Output Program 18
C. Pembahasan 22
D. Tautan Git 25
SOAL 2 26
A. Pembahasan 26
ADDITIONAL: 30

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Pilihan Persentase Tip 6
Gambar 3. Tampilan Aplikasi Setelah Dijalankan 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Awal) 18
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Pilihan Persentase) 18
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Sebelum Pembulatan) 19
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose (Setelah Pembulatan) 19
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML (Tampilan Awal) 20
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML (Pilihan Persentase) 20
Gambar 10. Screenshot Hasil Jawaban Soal 1 XML (Sebelum Pembulatan) 21
Gambar 11. Screenshot Hasil Jawaban Soal 1 XML (Setelah Pembulatan) 21

DAFTAR TABEL

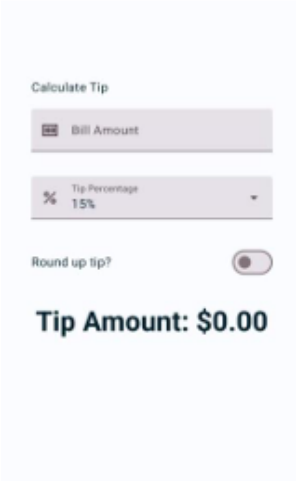
Tabel 1. Source Code Mainactivity Compose Soal 1 8
Tabel 2. Source Code Mainactivity XML Soal 1..... 13
Tabel 3. Source Code main_activity.xml Soal 1..... 15

SOAL 1

Soal Praktikum:

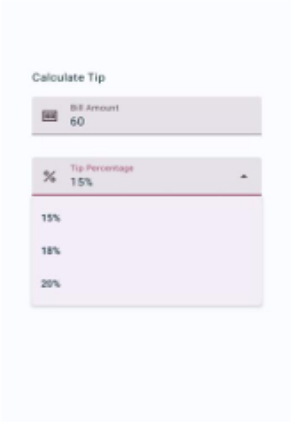
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.



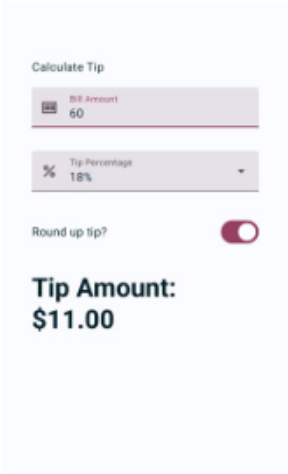
Gambar 1. Tampilan Awal Aplikasi

b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.



Gambar 2. Tampilan Pilihan Persentase Tip

c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt

MainActivity.kt Compose

Tabel 1. Source Code Mainactivity Compose Soal 1

```
package com.example.calculatortip

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import
androidx.compose.foundation.layout.safeDrawingPadding
import androidx.compose.foundation.layout.size
import
androidx.compose.foundation.layout.statusBarsPadding
import
androidx.compose.foundation.layout.wrapContentWidth
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import
androidx.compose.material3.OutlinedTextFieldDefaults
import androidx.compose.material3.Surface
import androidx.compose.material3.Switch
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
```

```
39 import androidx.compose.runtime.mutableStateOf
40 import androidx.compose.runtime.remember
41 import androidx.compose.runtime.setValue
42 import androidx.compose.ui.Alignment
43 import androidx.compose.ui.Modifier
44 import androidx.compose.ui.res.painterResource
45 import androidx.compose.ui.res.stringResource
46 import androidx.compose.ui.text.input.ImeAction
47 import androidx.compose.ui.text.input.KeyboardType
48 import androidx.compose.ui.text.font.FontWeight
49 import androidx.compose.ui.tooling.preview.Preview
50 import androidx.compose.ui.unit.dp
51 import
   com.example.calculatortip.ui.theme.CalculatorTipTheme
52 import java.text.NumberFormat
53
54 class MainActivity : ComponentActivity() {
55     override fun onCreate(savedInstanceState: Bundle?) {
56         enableEdgeToEdge()
57         super.onCreate(savedInstanceState)
58         setContent {
59             CalculatorTipTheme() {
60                 Surface(
61                     modifier = Modifier.fillMaxSize(),
62                 ) {
63                     CalculatorTipLayout()
64                 }
65             }
66         }
67     }
68 }
69
70 @Composable
71 fun CalculatorTipLayout() {
72     var amountInput by remember { mutableStateOf("") }
73     var tipInput by remember { mutableStateOf("") }
74     var roundUp by remember { mutableStateOf(false) }
75     var expanded by remember { mutableStateOf(false) }
76
77     val amount = amountInput.toDoubleOrNull() ?: 0.0
78     val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
79     val tip = calculateTip(amount, tipPercent, roundUp)
80     val options = listOf("15%", "18%", "20%")
81
82     Column(
83         modifier = Modifier
84             .statusBarsPadding()
85             .padding(horizontal = 40.dp)
86             .verticalScroll(rememberScrollState())
```

```
            .safeDrawingPadding(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp, top = 40.dp)
                .align(alignment = Alignment.Start)
        )
        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.baseline_money_24,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth(),
        )
        Box(modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()) {
            TextField(
                value = if (tipInput.isEmpty()) "" else "$tipInput%",
                onValueChange = { },
                readOnly = true,
                label = { Text(stringResource(R.string.how_was_the_service)) },
                leadingIcon = {
                    Icon(painter = painterResource(id = R.drawable.baseline_percent_24), null)
                },
                trailingIcon = {
                    IconButton(onClick = { expanded = true }) {
                        Icon(painter = painterResource(id = android.R.drawable.arrow_down_float), null)
                    }
                },
                modifier = Modifier.fillMaxWidth().clickable { expanded = true }
            )
```

```

            DropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false },
                modifier = Modifier.fillMaxWidth(0.6f)
            ) {
                options.forEach { option ->
                    DropdownMenuItem(
                        text = { Text(option) },
                        onClick = {
                            tipInput = option.replace("%", "")
                            expanded = false
                        }
                    )
                }
            }
        }
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall.copy (
                fontWeight = FontWeight.Bold
            )
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
        singleLine = true,
        leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },
```

```
        modifier = modifier,
        onValueChange = onValueChanged,
        label = { Text(stringResource(label)) },
        keyboardOptions = keyboardOptions
    )
}

@Composable
fun RoundTheTipRow(
    roundUp: Boolean,
    onRoundUpChanged: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .size(48.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = stringResource(R.string.round_up_tip))
        Switch(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentWidth(Alignment.End),
            checked = roundUp,
            onCheckedChange = onRoundUpChanged
        )
    }
}

private fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance(java.util.Locale.US).format(tip)
}

@Preview(showBackground = true)
@Composable
fun CalculatorTipPreview() {
    CalculatorTipTheme() {
            CalculatorTipLayout()
    }
}
```

MainActivity.kt XML

Tabel 2. Source Code Mainactivity XML Soal 1

```
package com.example.calculatortip

import android.os.Bundle
import android.widget.*
import android.text.Editable
import android.text.TextWatcher
import android.view.View
import androidx.appcompat.app.AppCompatActivity
import
com.google.android.material.textfield.TextInputEditText
import java.text.NumberFormat
import java.util.*

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main_activity)

        val editBillAmount:
com.google.android.material.textfield.TextInputEditText =
            findViewById(R.id.editBillAmount)
        val spinnerTipOptions: android.widget.Spinner =
findViewById(R.id.spinnerTipOptions)
        val switchRoundUp:
androidx.appcompat.widget.SwitchCompat =
findViewById(R.id.switchRoundUp)
        val textTipResult: android.widget.TextView =
findViewById(R.id.textTipResult)

        fun triggerUpdate() {
            calculateTip(editBillAmount,
spinnerTipOptions, switchRoundUp, textTipResult)
        }


        spinnerTipOptions.onItemSelectedListener = object
: AdapterView.OnItemSelectedListener {
            override fun onItemSelected(
                parent: AdapterView<*>?,
                view: android.view.View?,
                position: Int,
                id: Long
            ) {
                triggerUpdate()
            }
```

```

            override fun onNothingSelected(parent: AdapterView<*>?) {}
        }

        switchRoundUp.setOnCheckedChangeListener { _, _ ->
            triggerUpdate()
        }
        editBillAmount.addTextChangedListener(object : TextWatcher {
            override fun afterTextChanged(s: Editable?) {
                triggerUpdate()
            }

            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
        })
    }

    private fun calculateTip(
        editBillAmount: TextInputEditText,
        spinnerTipOptions: Spinner,
        switchRoundUp: androidx.appcompat.widget.SwitchCompat,
        textTipResult: TextView
    ) {
        val stringInTextField = editBillAmount.text.toString()
        val cost = stringInTextField.toDoubleOrNull()

        //jika input kosong
        if (cost == null) {
            textTipResult.text = "Tip Amount: $0.00"
            return
        }
        val tipPercentage = when (spinnerTipOptions.selectedItemPosition) {
            0 -> 0.15
            1 -> 0.18
            else -> 0.20
        }
        var tip = tipPercentage * cost

        if (switchRoundUp.isChecked) {
            tip = kotlin.math.ceil(tip)
        }
```

```

        val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        textTipResult.text = "Tip Amount: $formattedTip"
    }
}
```

activity_main.xml

Tabel 3. Source Code activity_main.xml Soal 1

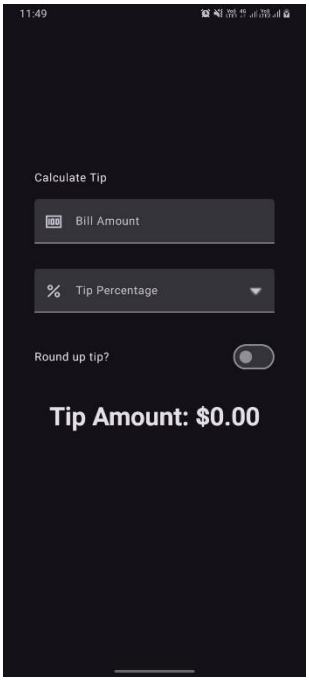
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp"
    android:background="?android:attr/windowBackground">

    <TextView
        android:id="@+id/textTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="40dp"
        android:text="Calculate Tip"
      android:textColor="?android:attr/textColorPrimary"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <LinearLayout
        android:id="@+id/billContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:gravity="center_vertical"
        android:orientation="horizontal"
app:layout_constraintTop_toBottomOf="@+id/textTitle">

        <ImageView
            android:id="@+id/iconMoney"
            android:layout_width="48dp"
            android:layout_height="48dp"
            android:padding="12dp"
            android:src="@drawable/baseline_money_24"
```

```
            app:tint="?android:attr/textColorPrimary" />

  <com.google.android.material.textfield.TextInputLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox">

<com.google.android.material.textfield.TextInputEditText
                android:id="@+id/editBillAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Bill Amount"
                android:inputType="numberDecimal"
android:textColor="?android:attr/textColorPrimary"/>
</com.google.android.material.textfield.TextInputLayout>
    </LinearLayout>

    <TextView
        android:id="@+id/labelTipOptions"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="Tip Percentage"
    android:textColor="?android:attr/textColorSecondary"
        android:textSize="16sp"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/billContainer"
/>

    <LinearLayout
        android:id="@+id/spinnerContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:orientation="horizontal"
        android:gravity="center_vertical"
app:layout_constraintTop_toBottomOf="@+id/labelTipOptions">

        <ImageView
            android:id="@+id/iconPercent"
            android:layout_width="48dp"
            android:layout_height="48dp"
            android:padding="12dp"
            android:src="@drawable/baseline_percent_24"
            app:tint="?android:attr/textColorPrimary" />

```

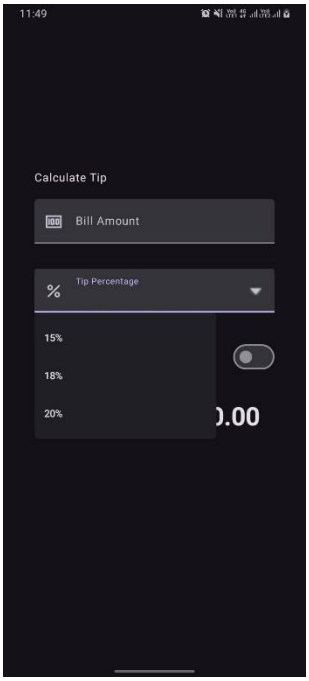
```
        <Spinner
            android:id="@+id/spinnerTipOptions"
            android:layout_width="0dp"
            android:layout_height="48dp"
            android:layout_weight="1"
            android:entries="@array/tip_options" />
    </LinearLayout>

    <TextView
        android:id="@+id/labelRoundUp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="Round up tip?"
      android:textColor="?android:attr/textColorPrimary"
        android:textSize="18sp"
        app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@+id/spinnerContainer" />

    <androidx.appcompat.widget.SwitchCompat
        android:id="@+id/switchRoundUp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
app:layout_constraintBottom_toBottomOf="@+id/labelRoundUp"
        app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="@+id/labelRoundUp" />

    <TextView
        android:id="@+id/textTipResult"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="40dp"
        android:text="Tip Amount: $0.00"
      android:textColor="?android:attr/textColorPrimary"
        android:textSize="28sp"
        android:textStyle="bold"
        app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toBottomOf="@+id/labelRoundUp" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

B. Output Program

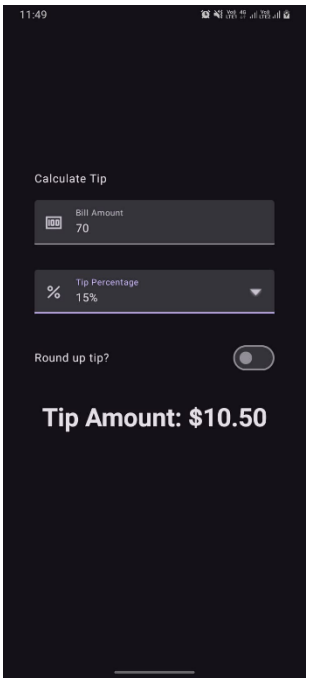
A. Compose



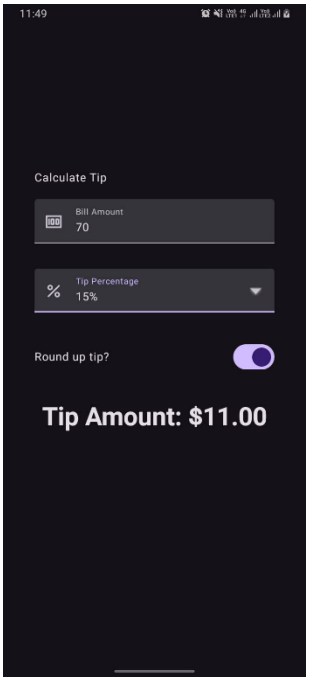
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Tampilan Awal)



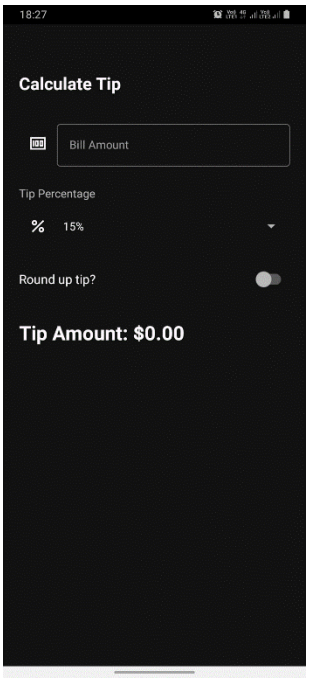
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Pilihan Persentase)



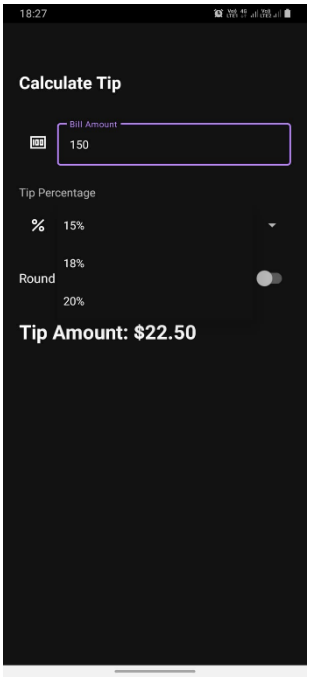
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Sebelum Pembulatan)



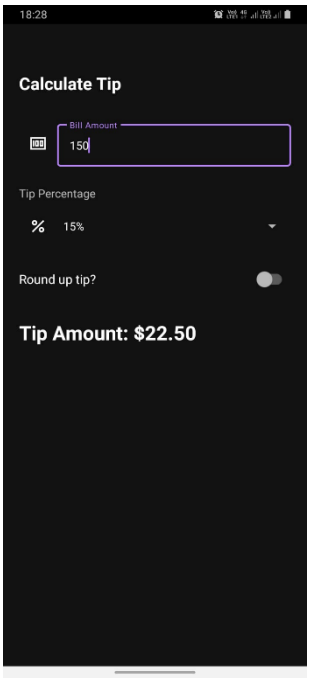
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose (Setelah Pembulatan)



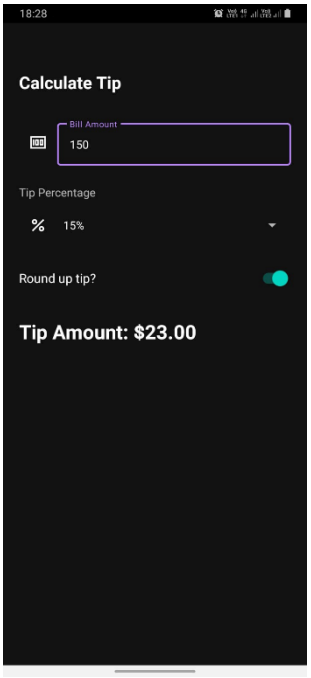
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML (Tampilan Awal)



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML (Pilihan Persentase)



Gambar 10. Screenshot Hasil Jawaban Soal 1 XML (Sebelum Pembulatan)



Gambar 11. Screenshot Hasil Jawaban Soal 1 XML (Setelah Pembulatan)

C. Pembahasan

MainActivity.kt Compose:

Dalam Kotlin, penulisan "package name" dan "import library" digunakan untuk mengorganisasi kode serta meminimalkan konflik nama kelas. Penulisan kelas diawali dengan keyword "class" diikuti nama kelasnya. Pada kode ini dilakukan *extends* ke *ComponentActivity* sebagai basis aktivitas Jetpack Compose. Penulisan method atau fungsi harus menggunakan keyword "fun" yang diikuti dengan nama fungsinya. Method "onCreate" digunakan sebagai titik masuk utama aplikasi untuk menyiapkan tata letak melalui fungsi "setContent" yang mengatur tema dan struktur antarmuka secara deklaratif.

Variabel di Kotlin dibedakan menjadi dua tipe, yakni *val* untuk nilai konstan dan *var* untuk nilai yang bersifat dinamis atau dapat berubah. Dalam aplikasi kalkulator tip ini, keyword var digunakan pada bagian state agar UI dapat diperbarui secara otomatis saat pengguna memasukkan input jumlah uang. Terdapat pula penggunaan operator *safe call* (?) dan *elvis operator* (?:) yang berfungsi untuk memberikan nilai default apabila variabel bernilai null. Hal ini sangat penting untuk mencegah terjadinya *Null Pointer Exception (NPE)* saat melakukan konversi tipe data dari teks input menjadi angka. Penggunaan sistem variabel yang tepat memastikan aplikasi berjalan stabil meskipun menerima input yang tidak valid dari pengguna.

Logika antarmuka dibangun menggunakan fungsi "@Composable" dengan komponen *Column* dan *Row* untuk mengatur posisi elemen secara vertikal maupun horizontal. Penggunaan fungsi "remember" dan "mutableStateOf" sangat krusial dalam menyimpan status aplikasi selama proses *recomposition* agar data yang diinput pengguna tidak hilang. Selain itu, terdapat fungsi calculateTip yang bertugas melakukan perhitungan matematis dan dilengkapi logika pembulatan menggunakan "kotlin.math.ceil". Komponen *Material Design 3* diterapkan dengan bantuan *Modifier* untuk mengatur tata letak, ukuran, hingga jarak antar elemen agar terlihat rapi. Seluruh elemen string dan gambar diakses melalui sistem resource Android menggunakan anotasi "@StringRes" dan "@DrawableRes" untuk mempermudah pengelolaan aset.

MainActivity.kt XML:

Dalam pemrograman Android menggunakan Kotlin, penulisan "package" dan "import" berfungsi untuk mengorganisasi file serta memanggil komponen library seperti *Widget* dan *View*. Pada kode ini, kelas MainActivity melakukan *extends* ke AppCompatActivity, yang merupakan standar dasar untuk aplikasi Android berbasis XML agar tetap kompatibel pada berbagai versi perangkat. Fungsi "onCreate" digunakan sebagai titik masuk utama untuk menginisialisasi aktivitas dan menghubungkan logika kode dengan desain antarmuka melalui perintah "setContentView(R.layout.main_activity)". Setiap komponen UI seperti "TextInputEditText", "Spinner", dan "Switch" dideklarasikan menggunakan "findViewById" agar dapat diakses dan dimanipulasi melalui kode program secara langsung.

Interaksi antar elemen dalam aplikasi ini dikelola menggunakan *Listener* agar aplikasi dapat merespons input pengguna secara real-time. Untuk menangkap perubahan teks pada kolom jumlah tagihan, digunakan fungsi "addTextChangedListener" yang secara otomatis akan memicu pembaruan hasil setiap kali pengguna mengetik. Selain itu, terdapat "onItemSelectedListener" pada komponen *Spinner* dan "setOnCheckedChangeListener" pada komponen Switch untuk memantau pilihan persentase tip dan status pembulatan. Penggunaan fungsi pembantu "triggerUpdate" di dalam listener tersebut memastikan bahwa setiap perubahan kecil pada input akan langsung diproses oleh fungsi logika tanpa perlu menekan tombol tambahan.

Logika perhitungan utama berada di dalam fungsi calculateTip, di mana input teks terlebih dahulu dikonversi menjadi tipe data angka menggunakan "toDoubleOrNull". Untuk mencegah terjadinya error saat input kosong, digunakan pengecekan kondisi agar sistem memberikan nilai default secara otomatis jika data yang dimasukkan tidak valid. Penentuan besar tip dilakukan dengan ekspresi *when* yang memilih nilai berdasarkan posisi *index* pada *Spinner*, yang kemudian dikalikan dengan total biaya. Hasil perhitungan tersebut dapat dibulatkan ke atas menggunakan

"kotlin.math.ceil" jika fitur pembulatan aktif, lalu diformat ke dalam mata uang dolar melalui "NumberFormat" agar tampilan akhirnya terlihat lebih profesional dan informatif bagi pengguna.

main_activity.xml:

main_activity.xml adalah file XML yang digunakan untuk tata letak aktivitas utama. File ini menggunakan kombinasi antara ConstraintLayout sebagai kontainer utama dan LinearLayout untuk menyusun beberapa elemen secara horizontal. Berikut adalah rincian komponennya:

1. Deklarasi XML dan Namespace:
   - Baris "<?xml version="1.0" encoding="utf-8"?>" merupakan *XML Declaration* yang berfungsi memberi tahu sistem tentang cara membaca file ini.
   - "xmlns:android: " Merupakan deklarasi wajib agar atribut seperti android:id dikenali sebagai atribut resmi milik Android.
   - "xmlns:app: " Digunakan untuk atribut khusus dari *library* tambahan (seperti *ConstraintLayout* atau *Material Components*).
   - "xmlns:tools: " Digunakan untuk keperluan bantuan desain di Android Studio yang tidak akan muncul saat aplikasi dijalankan.
2. ConstraintLayout (Kontainer Utama): Adalah kontainer yang berfungsi untuk menyusun komponen UI dengan cara menghubungkan satu elemen ke elemen lainnya atau ke tepi layar (*Parent*). Pada kode ini, *ConstraintLayout* digunakan agar tata letak aplikasi tetap responsif dan tidak berantakan di berbagai ukuran layar melalui atribut seperti "app:layout_constraintTop_toTopOf" dan "app:layout_constraintStart_toStartOf".
3. LinearLayout (Kontainer Baris): Adalah sebuah kontainer atau wadah yang berfungsi untuk menyusun komponen UI di dalamnya secara berurutan dalam satu garis lurus. Dalam kode ini, *LinearLayout* digunakan untuk menyusun "ImageView" dan "TextInputLayout" secara horizontal agar sejajar dalam satu baris.

4. Elemen-elemen UI:
    - "TextView": Digunakan untuk menampilkan teks di layar, seperti judul "Calculate Tip" dan hasil akhir "Tip Amount". Elemen ini menggunakan atribut "android:textSize" untuk ukuran huruf dan "android:textStyle="bold"" untuk menebalkan teks.
    - "ImageView": Berfungsi untuk menampilkan gambar atau ikon di aplikasi. Atribut android:src digunakan untuk mengambil ikon money dan percent dari folder res/drawable.
    - "TextInputLayout" & "TextInputEditText": Merupakan komponen dari Material Design untuk menangkap input angka dari pengguna. Atribut "android:inputType="numberDecimal"" memastikan agar keyboard yang muncul hanya angka dan titik desimal.
    - "Spinner": Berfungsi sebagai elemen interaktif yang menampilkan daftar pilihan persentase tip (*drop-down*). Pilihan kontennya diambil melalui atribut android:entries.
    - "SwitchCompat": Berfungsi sebagai elemen interaktif yang bisa digeser oleh pengguna untuk mengaktifkan atau mematikan fitur pembulatan tip (round up).

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat:
https://github.com/AnggraeniDwiZahra/Pemrograman-Mobile

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

XML *(eXtensible Markup Language)* adalah bahasa markup yang digunakan untuk menyimpan dan mentransfer data. Saat mengembangkan aplikasi Android, XML digunakan untuk mendefinisikan tata letak dan komponen Antarmuka Pengguna (UI). XML menjelaskan format dan elemen visual UI, termasuk penempatan tombol, tampilan teks, dan elemen UI lainnya. Di sisi lain, Jetpack Compose adalah toolkit modern yang direkomendasikan Android untuk membangun UI native. Jetpack Compose menyederhanakan dan mempercepat pengembangan UI di Android.

Berikut adalah perbandingan antara XML dan Jetpack Compose:

Aspek	Jetpack Compose	Arsitektur XML
Pendekatan terhadap desain dan struktur UI	Kontemporer, deklaratif, dan dapat disusun.	Imperatif dan hierarkis
Deskripsi UI	Didefinisikan menggunakan fungsi Kotlin	Ditentukan dalam file XML
Reaktivitas	Pendekatan reaktif berdasarkan status aplikasi.	Membutuhkan pengelolaan manual untuk perubahan dan pembaruan status tampilan.
Dapat digunakan kembali	Elemen UI yang dapat disusun sebagai blok yang dapat digunakan kembali.	Lihat hierarki untuk pengaturan tata letak.
Interaksi dan Animasi UI	Pengelolaan yang lebih mudah dan nyaman melalui fungsi yang dapat dikombinasikan.	Penanganan manual, kode yang lebih kompleks dan bertele-tele.
Modularitas dan	Enkapsulasi perilaku dalam	Kode dapat semakin sulit

Kemudahan Pemeliharaan	fungsi-fungsi yang dapat disusun secara khusus.	dipelihara seiring berkembangnya aplikasi.
Kemampuan uji	Memungkinkan kode yang lebih bersih dan lebih mudah diuji.	Memerlukan lebih banyak upaya dalam pengujian.
Kurva Pembelajaran	Memiliki kurva pembelajaran yang lebih curam bagi pengembang yang baru mengenal UI deklaratif.	Cocok untuk pengembang yang sudah terbiasa dengan XML dan pendekatan imperatif.
Pertunjukan	Performa yang efisien dan berpotensi lebih baik berkat teknik optimasi UI.	Dapat menimbulkan beban kinerja tambahan pada tata letak yang besar dan kompleks.
Komunitas Pembangunan	Komunitas yang berkembang dan dukungan pengembangan aktif	Komunitas yang matang dan mapan dengan sumber daya yang melimpah.
Dukungan IDE	Dukungan IDE yang baik dengan Android Studio.	Dukungan terintegrasi penuh dengan Android Studio.

Kelebihan dan Kekurangan

a. **Kelebihan dan Kekurangan Jetpack Compose**

Kelebihan dari Jetpack Compose adalah merupakan bagian dari Android Jetpack yang terintegrasi langsung dengan ekosistem android. Sehingga jika ingin membangun aplikasi khusus android, pada umumnya disarankan menggunakan Jetpack Compose.

Kekurangan jetpack compose:

- Kurva Pembelajaran: Membutuhkan konsep dan paradigma baru yang cukup kompleks, terutama bagi yang tidak terbiasa dengan pemrograman deklaratif.
- Dokumentasi dan Dukungan Komunitas: Sebagai pendatang baru, dukungan komunitas dan dokumentasi mungkin kurang luas, sehingga sulit menemukan solusi untuk masalah tertentu.

- Kematangan dan Stabilitas: Belum menawarkan tingkat kematangan dan stabilitas yang sama seperti kerangka kerja UI yang sudah mapan.
- Pertimbangan Kinerja: UI yang kompleks dengan *composable* bertingkat atau *layout* inefisien dapat memengaruhi kinerja aplikasi.
- Kompatibilitas Pihak Ketiga: Integrasi dengan pustaka pihak ketiga atau tampilan khusus Android tradisional dapat menimbulkan tantangan kompatibilitas.
- Dukungan Perangkat Pengembangan: Ekosistem perangkat pengembangan masih berkembang, sehingga mungkin ada keterbatasan dibandingkan kerangka kerja UI yang sudah mapan.
- Adopsi dan Dukungan Jangka Panjang: Tingkat adopsi bervariasi; kekhawatiran tentang komitmen jangka panjang Google dapat memengaruhi keputusan penggunaan dalam proyek produksi.

b. Kelebihan dan Kekurangan XML

Kelebihan

- XML merupakan platform dan bahasa yang independen, sehingga dapat digunakan pada berbagai sistem.
- Mendukung unicode, yaitu standar pengkodean internasional untuk digunakan dengan berbagai bahasa dan aksara. setiap huruf, angka, atau simbol akan diberi nilai numerik unik yang berlaku di berbagai platform dan program. Sehingga XML dapat mengirimkan informasi apa pun yang ditulis dalam bahasa manusia apa pun.
- Dapat merubah data kapan saja tanpa mempengaruhi tampilannya.
- XML memungkinkan validasi menggunakan DTD dan Skema. Validasi ini memastikan bahwa dokumen XML bebas dari kesalahan sintaksis.
- XML menyederhanakan berbagi data antar berbagai sistem karena sifatnya yang independen terhadap platform. Data XML tidak memerlukan konversi apa pun saat ditransfer antar sistem yang berbeda.

Kekurangan

- Tidak mendukung array.
- Ukuran cukup besar, tergantung pada siapa yang menulisnya.
- Biaya penyimpanan dan pengiriman data cukup tinggi karena adanya Redundansi dalam sintaks XML
- Sintaks XML bertele-tele dan berlebihan dibandingkan dengan format transmisi data berbasis teks lainnya
- Dokumen XML kurang mudah dibaca dibandingkan dengan format transmisi data berbasis teks lainnya

Additional:

ESSAY

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

   Berdasarkan literatur "Kotlin in Action", Kotlin memiliki berbagai fitur yang dirancang untuk produktivitas dan keamanan kode, antara lain:

   - *Null Safety*: Kotlin menggunakan sistem tipe data yang eksplisit untuk nilai yang boleh kosong **(Int?)**, sehingga membantu pengembang menghindari kesalahan fatal *NullPointerException* [1].
   - Ringkas (*Concise*): Pengembang dapat mendefinisikan *Data Class* beserta propertinya tanpa perlu menulis isi body kelas secara manual [1].
   - *Read-only Properties*: Penggunaan kata kunci **val** memastikan properti hanya bisa dibaca (konstan), yang mendukung konsep *immutability* [1].
   - Sintaks Modern: Dukungan terhadap *string templates*, *named arguments*, *default values*, hingga penggunaan *trailing commas* membuat kode lebih bersih [1].
   - Fungsi Tingkat Atas: Definisi fungsi dapat dilakukan di tingkat atas tanpa harus selalu dibungkus di dalam kelas [1].

   Selain itu, Kotlin juga menyediakan dukungan kaya untuk gaya pemrograman fungsional yang menjadi daya jualnya :

   - *Lambda Expressions*: Memungkinkan pengiriman blok kode dengan boilerplate minimal [1].
   - *Collection Operations*: Tersedianya API standar seperti *maxByOrNull* memudahkan pengolahan data koleksi secara fungsional [1].
   - *Elvis Operator* (**?:**): Menyediakan nilai cadangan secara ringkas jika sebuah variabel bernilai null [1].

Berdasarkan penelitian dalam Jurnal KDI, disampaikan bahwa Kotlin dirancang untuk mengatasi keterbatasan Java dengan menawarkan sintaks yang lebih efisien dan fitur modern seperti *extension functions* serta *coroutines* [2]. Meskipun Java terkadang unggul dalam performa mentah pada beberapa benchmark, perbedaan tersebut tidak signifikan jika dibandingkan dengan kemudahan pemeliharaan kode yang ditawarkan Kotlin. Penelitian lain menunjukkan bahwa 61% alasan pemilihan bahasa modern seperti Kotlin adalah karena sintaks yang ringkas, sementara 46% lainnya didorong oleh fitur keamanan seperti null safety [3].

2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Soal/Skenario:

Memastikan bahwa fitur *Data Class* dapat menduplikasi objek dengan perubahan tertentu tanpa merusak data pada objek aslinya (*Immutability Check*).

Langkah-langkah:

1. Arrange: Buat objek UserAsli dengan ID 1 dan Nama "Budi".
2. Act: Gunakan fungsi .copy() untuk membuat UserBaru dengan mengubah namanya menjadi "Andi".
3. Assert: Verifikasi bahwa nama pada objek baru telah berubah, namun data pada objek asli tetap tidak bergeser.

Penyelesaian Kode (Unit Test):

```
@Test
 fun testDataClassCopy() {
    // 1. Persiapan data
    val userAsli = User(id = 1, nama = "Budi")

    // 2. Eksekusi duplikasi dengan perubahan
    val userBaru = userAsli.copy(nama = "Andi")
```

```
    // 3. Verifikasi hasil
    assertEquals("Andi", userBaru.nama) // Objek baru
    assertEquals(1, userBaru.id)        // ID objek baru
    assertEquals("Budi", userAsli.nama) // Objek asli
}
```

DAFTAR PUSTAKA

[1] S. Isakova, R. Elizarov, S. Aigner, and D. Jemerov, *Kotlin in Action , Second Edition MEAP V09*, 2nd ed. Manning Publications, 2024.

[2] J. J. Sanjaya and J. Susilo, “Perbandingan Performa Kotlin vs Java dalam Pengembangan Android dengan Metode Iterasi While,” *bit-Tech*, vol. 7, no. 2, pp. 545–553, 2024, doi: 10.32877/bt.v7i2.1898.

[3] Y. Biantoro, Yasmiati, R. Ulansari, Jenih, R. B. Styanto, and Suharyanto, “Perkembangan Bahasa Pemrograman Terkini: Kajian Literatur Sistematis Terhadap Preferensi, Kinerja, dan Penggunaan di Dunia Industri dan Akademik,” vol. 11, no. 2, pp. 255–267, 2025, [Online]. Available: https://ejournal.urindo.ac.id/index.php/TI/article/download/7466/2483